

Lecture 5: Pandas

CMSC/DATA 320: Introduction to Data Science

Lecturer: Anna Evtushenko (annae@umd.edu)

Assignments

- PA1 and WA1 were due yesterday
- WA2, PA2 released tomorrow
- Quiz 2 has been graded; grades will be published today/tomorrow

Important tools

- **Python:** A user-friendly language for data analysis.
- **Git:** Helps manage code and data changes when working with a team.
- **Pandas:** Simplifies data cleaning and manipulation.
- **Databases:** Needed for storing and retrieving data efficiently.

Knowing how to use these is essential for effective data analysis, collaboration, and handling data in real-world projects.

Basic Concept Review: Table / Tabular Data

In tabular data, information is organized into rows and columns.

Special Column, called “Index”, or
“ID”, or “Key”
Usually, no duplicates Allowed

Variables
(also called Attributes, or
Columns, or Labels)

Observations,
Rows, or
Tuples

ID	age	wgt_kg	hgt_cm
1	12.2	42.3	145.1
2	11.0	40.8	143.8
3	15.6	65.3	165.3
4	35.1	84.2	185.8

The diagram illustrates the components of a table. A purple arrow points from the text 'Special Column, called “Index”, or “ID”, or “Key”' to the 'ID' column header. Three orange arrows point from the text 'Variables (also called Attributes, or Columns, or Labels)' to the 'age', 'wgt_kg', and 'hgt_cm' column headers. Four blue arrows point from the text 'Observations, Rows, or Tuples' to the four data rows of the table.

Pandas

What is Pandas

"Pandas": Short for "**Python Data Analysis Library**," **used for data analysis**.

Written by Wes McKinney: Started in 2008 to get a high-performance, flexible tool to perform quantitative analysis on financial data.

- ❏ An open-source Python library that provides high-performance, easy-to-use data structures and data analysis tools.
- It's built on **top of NumPy** and is particularly useful for working with structured (tabular) data.
- Key Data Structures
 - a. Series
 - b. DataFrame

Why Pandas?

- Easy handling of **missing data**.
- Powerful **grouping** and **aggregation** functions.
- Quick **data filtering**, **sorting**, and **merging**.
- Excellent support for **reading/writing data** (CSV, Excel, SQL, JSON).

Pandas First Steps: **install** and **import**

- Pandas is an easy package to install. Open up your terminal program (shell or cmd) and install it using either of the following commands:

```
$ conda install pandas  
OR  
$ pip install pandas
```

- For **jupyter notebook** users, you can run this cell:
 - The **!** at the beginning runs cells as if they were in a terminal.
- For Google Colab: already pre-installed; just import it
- To import pandas we usually import it with a shorter name since it's used so much:

```
!pip install pandas
```

```
import pandas as pd
```

Next: Import the Pandas package

Once Pandas is installed, import it in your applications by adding the **import** keyword:

```
In [ ]: import pandas as pd
```

“pd” → **alias** (in Python): an alternate name for referring to the same thing .

```
In [ ]: pd?
```

Now Pandas is imported and ready to use!

Core components of Pandas:

Series & DataFrames

Pandas: Data Table Representation

Data in Pandas is stored in two fundamental data types:

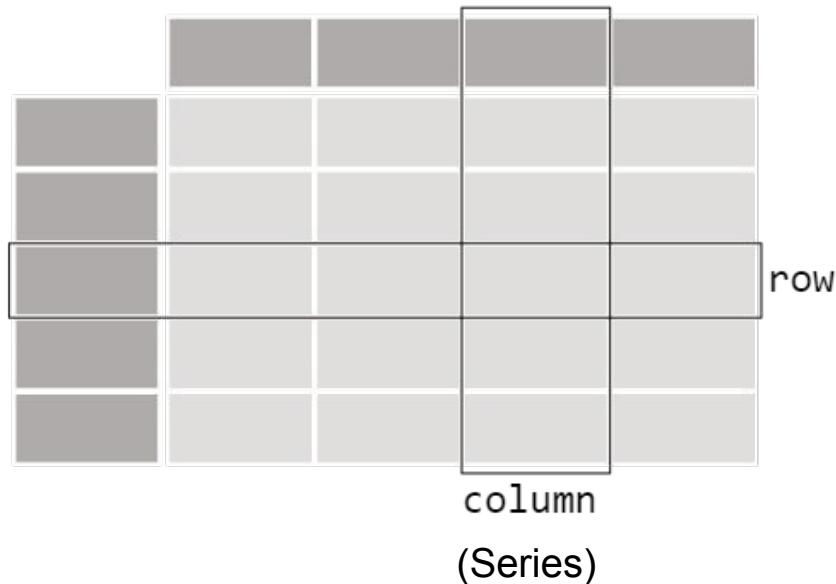
- **Series** is the data type that stores a single column of data.
- **DataFrame** is the data type that stores an entire dataset.

Notes: They both behave very similar to numpy arrays, so you can use them as input to anything in numpy, scipy, or scikit-learn.

Pandas: Data Table Representation cont.

In pandas, the most basic object is the DataFrame, [that is a collection of Series objects](#). Each column in a DataFrame is essentially a Series.

DataFrame



The primary two components of pandas are the Series and DataFrame.

- **Series** is essentially a column, and
- **DataFrame** is a multi-dimensional table made up of a collection of Series.

Series

1. Series

One-dimensional labeled array (like a column).

- Similar to NumPy array but with an explicit index column.

```
import pandas as pd

# Create a Series
s = pd.Series([1, 3, 5, 7, 9])
print(s)
```

Create a Series

`pd.Series([1, 2, 2, np.nan], index=['p', 'q', 'r', 's'])`

Data	
Index	p
	1.0
	q
	2.0
	r
	2.0
	s
	NaN

dtype: float64

© w3resource.com

index values

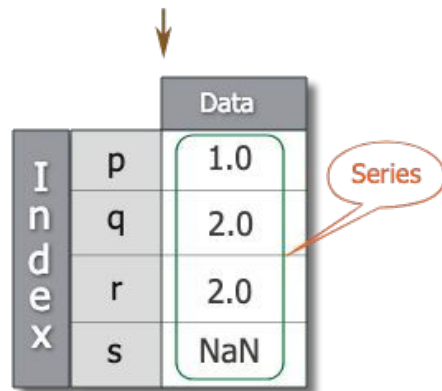
A	→	5
B	→	6
C	→	12
D	→	-5
E	→	6.7

Series

A one-dimensional array of data with an index

- Subclass of `numpy.ndarray`
- Has two components: index and value for each element
- **Data:** any type, but has to be the same for all elements
- **Index:** most commonly integers, start with 0 (default).
 - Index labels need not be ordered but it's best if they are

```
pd.Series([1, 2, 2, np.nan], index=['p', 'q', 'r', 's'])
```



Data	
Index	1.0
p	2.0
q	2.0
r	NaN
s	

dtype: float64

© w3resource.com

index **values**

A	→	5
B	→	6
C	→	12
D	→	-5
E	→	6.7

Dataframes

The primary pandas data structure that has a tabular format similar to Excel

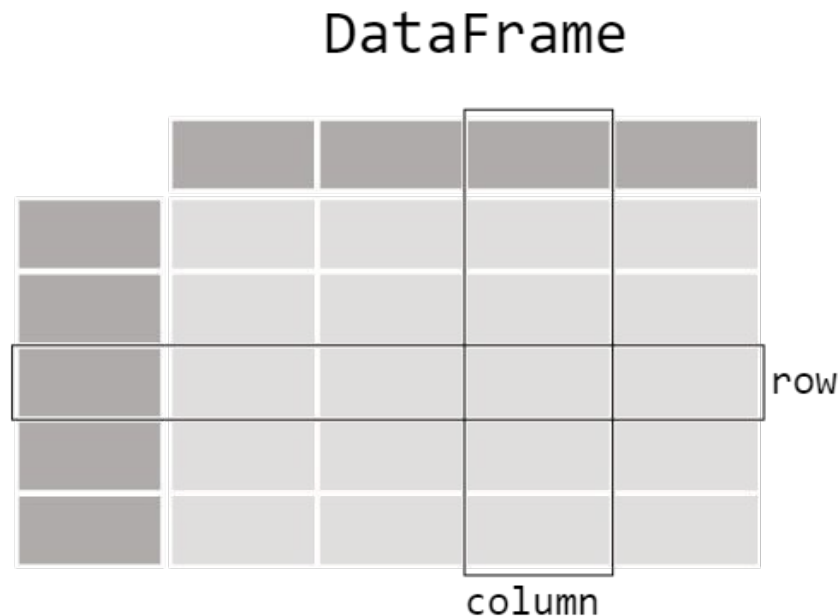
2. Dataframe

Two-dimensional labeled data structure/table (like a spreadsheet or SQL table).

Most commonly used Pandas object.

```
# Create a DataFrame
data = {'Name': ['Alice', 'Bob', 'Charlie'],
        'Age': [25, 30, 35],
        'City': ['NY', 'LA', 'Chicago']}
df = pd.DataFrame(data)
print(df)
```

Create a DataFrame



Dataframes

A collection of series organized in columns.

- Each column can have a different type (integers, floats, strings, etc.)
- We can add and remove columns
- Can Perform Arithmetic operations on rows and columns
- We do not access rows via index (less common).
- We **DO** access columns via their names

The diagram shows a table with 7 rows and 5 columns. The columns are labeled 'Name', 'Team', 'Number', 'Position', and 'Age'. The rows are indexed 0 to 6. A label 'Columns' points to the column headers. A label 'Rows' points to the row indices. A label 'Data' points to the data cells. A small logo is in the bottom right corner.

	Name	Team	Number	Position	Age
0	Avery Bradley	Boston Celtics	0.0	PG	25.0
1	John Holland	Boston Celtics	30.0	SG	27.0
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN
6	Evan Turner	Boston Celtics	11.0	SG	27.0

The diagram shows two Series being added together to form a DataFrame. The first Series is 'apples' with values [3, 2, 0, 1]. The second Series is 'oranges' with values [0, 3, 7, 2]. The resulting DataFrame has columns 'apples' and 'oranges' and rows indexed 0 to 3. Arrows indicate the relationship between the Series and the DataFrame, and between the DataFrame and its rows and columns.

	apples	oranges
0	3	0
1	2	3
2	0	7
3	1	2

Dataframe Functions

Important functions:

- How to access a column
- How to filter a column
- Apply
- GroupBy

Basic Operations: Reading Data

Read CSV file

```
df = pd.read_csv('data.csv')
```

Read Excel file

```
df = pd.read_excel('data.xlsx') [ Reading (both XLS and XLSX), using the file path  
as an input.]
```

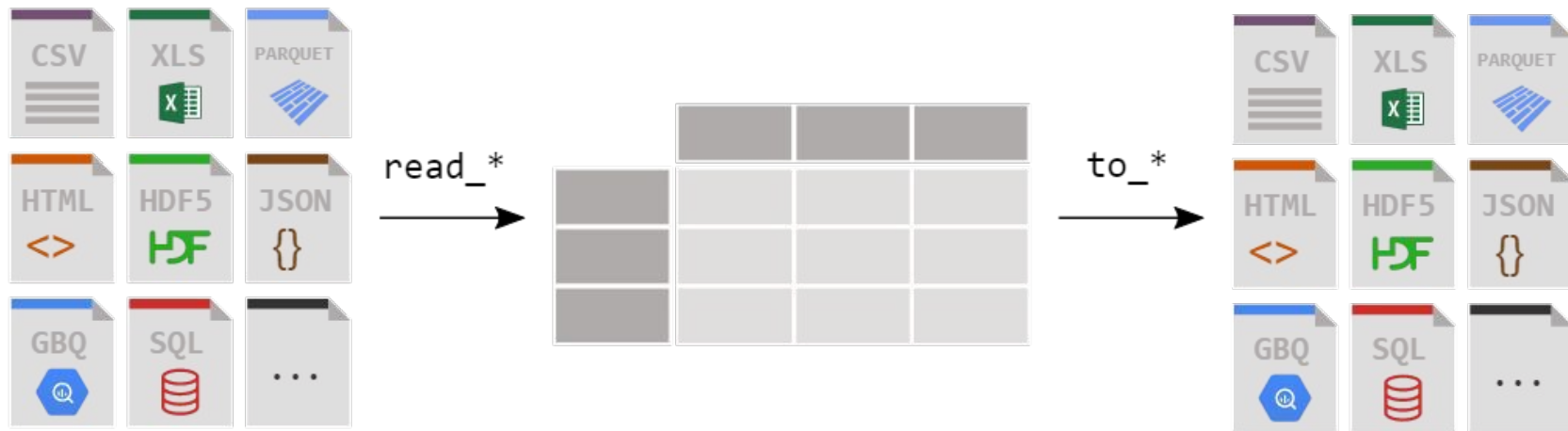
Read Text file

```
df = pd.read_csv("diabetes.txt", sep="\s") [sep argument - separator. sep="\s"  
referring to single whitespace]
```

Read Json file

```
df = pd.read_json("diabetes.json")
```

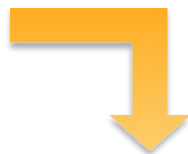
Loading a DataFrame from files



Load a CSV file into a Pandas DataFrame:

	A	B	C	D
1		apples	oranges	
2	Ahmad	3	0	
3	Ali	2	3	
4	Rashed	0	7	
5	Hamza	1	2	
6				

Reading data from a CSV file



```
1 import pandas as pd
2
3 df = pd.read_csv('dataset.csv')
4 print(df)
5
```

- With CSV files, all you need is a single line to load in the data:

```
df = pd.read_csv('dataset.csv')
```



	Unnamed: 0	apples	oranges
0	Ahmad	3	0
1	Ali	2	3
2	Rashed	0	7
3	Hamza	1	2

*** If the data contains only one column and you **specify squeeze=True**, pandas will convert the result to a Series.

Basic Operations: Outputting data

into a CSV file

```
df = df.to_csv("diabetes_out.csv", index=False)
```

For a Series s: `s.to_csv("filename.csv")`

into an Excel file

```
df.to_excel("diabetes_out.xlsx", index=False)
```

into a Text file

```
df = f.to_csv('diabetes_out.txt', header=df.columns, index=None, sep=''
```

') # into a JSON file

```
df.to_json("diabetes_out.json")
```

Viewing and understanding DataFrames using pandas

After reading tabular data as a DataFrame, you would need to have a glimpse of the data. You can either view a small sample of the dataset or a summary of the data in the form of summary statistics.

Example:

Use **DataFrame.head()** and **DataFrame.tail()** to view the top (first) and bottom (last) few rows of the frame respectively.

- Default value is **5**, unless specified otherwise.

Basic Operations: Exploring Data

# Display First 5 rows <code>df.head()</code> # Display Last 3 rows <code>df.tail(n=3)</code> [specify the number of rows through the n argument (the default value is 5)].	# Data types <code>df.dtypes</code> # Summary statistics <code>df.describe()</code> prints the summary statistics of all numeric columns, such as count, mean, standard deviation, range, and quartiles of numeric columns.	# Shape (rows, columns) <code>df.shape</code> # Column names <code>df.columns</code> # Overview of columns and types <code>df.info()</code>
--	--	---

Basic Operations: Selecting Data

<pre># Select column df['Age'] # Select multiple columns df[['Name', 'City']]</pre>	<pre># Select rows by index df.iloc[0] # First row df.iloc[1:3] # Rows 1-2</pre>	<pre># Select rows by condition df[df['Age'] > 30]</pre>
--	--	---

How to access a column

`dataframe['column_name']`: Access a column by its name.

E.g. : `df['Age']`

Where `df`=your dataframe

The diagram shows a DataFrame with 7 rows and 6 columns. The columns are labeled *Name*, *Team*, *Number*, *Position*, and *Age*. The rows are indexed 0 to 6. A label 'Columns' with arrows points to the column headers. A label 'Rows' with arrows points to the row indices. A purple box labeled 'Data' highlights a subset of the data, specifically the 'Number' and 'Position' columns for rows 2 through 6.

	<i>Name</i>	<i>Team</i>	<i>Number</i>	<i>Position</i>	<i>Age</i>
0	Avery Bradley	Boston Celtics	0.0	PG	25.0
1	John Holland	Boston Celtics	30.0	SG	27.0
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN
6	Evan Turner	Boston Celtics	11.0	SG	27.0

Creating new columns/Arithmetic Operations

Can **perform arithmetic operations** (e.g., addition, subtraction, multiplication, division) on entire columns or between columns.

E.g.:

```
df['new_column'] = df['column1'] +  
df['column2']  
df['new_column'] = df['column1'] -  
df['column2']
```

Applying Functions Directly to the dataframe

You can apply different functions directly to a DataFrame column:

Syntax: **dataframe[column_name].function()**

The diagram illustrates a DataFrame structure. A table with 7 rows and 6 columns is shown. The columns are labeled 'Name', 'Team', 'Number', 'Position', and 'Age'. The rows are indexed from 0 to 6. Annotations include: 'Columns' with arrows pointing to the column headers; 'Rows' with arrows pointing to the row indices; and 'Data' with a bracket pointing to the data cells. Specific cells are highlighted with red boxes: 'Jonas Jerebko' in the Name column of row 2, '8.0' in the Number column of row 2, 'Boston Celtics' in the Team column of row 2, 'PG' in the Position column of row 4, and 'NaN' in the Age column of row 5.

	Name	Team	Number	Position	Age
0	Avery Bradley	Boston Celtics	0.0	PG	25.0
1	John Holland	Boston Celtics	30.0	SG	27.0
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN
6	Evan Turner	Boston Celtics	11.0	SG	27.0

Applying Functions Directly to the dataframe

Aggregation operations are used to compute summary statistics or single values from multiple values in a dataset. E.g:

E.g.: `df["Age"].sum()`

The diagram illustrates a dataframe structure. A table with 7 rows and 5 columns is shown. The columns are labeled 'Name', 'Team', 'Number', 'Position', and 'Age'. The rows are indexed 0 to 6. Annotations include: 'Columns' with arrows pointing to the column headers; 'Rows' with arrows pointing to the row indices; and 'Data' with a bracket pointing to the data cells. Some cells are highlighted with pink boxes: 'Jonas Jerebko' in the Name column of row 2, '8.0' in the Number column of row 2, 'Boston Celtics' in the Team column of row 3, 'PG' in the Position column of row 4, and 'NaN' in the Age column of row 5.

	Name	Team	Number	Position	Age
0	Avery Bradley	Boston Celtics	0.0	PG	25.0
1	John Holland	Boston Celtics	30.0	SG	27.0
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN
6	Evan Turner	Boston Celtics	11.0	SG	27.0

Filtering in Pandas

Focus on **specific subsets** of your data

Two Types:

1. Column Filtering (Selecting Columns):

Select specific columns or filter columns based on names/conditions. Method: Basic Selection: `df[['col1']]` or `df[['col1', 'col2']]`.

- Preparing data for modeling (selecting features).
- Reducing memory usage by dropping unused columns.

2. Row Filtering (Selecting Rows):

Filter rows based on conditions applied to column values. Method: **Boolean Indexing** (Most common one).

- Analyzing subsets (e.g., specific time periods, categories).
- Cleaning data (removing duplicates, outliers).

Count Unique Values

Basic Usage: Count occurrences of each unique value in a column:

Returns the frequency of each unique value in a Series, sorted by default in descending order.

```
# Count how many times each species appears
species_counts = flowers['species'].value_counts()
print(species_counts)
```

```
setosa      50
versicolor  50
virginica   50
Name: species, dtype: int64
```

Use `value_counts()` after filtering when you want to see/count how many times each unique value appears in a Series.

Key Aggregation Methods Supported by Pandas

Method	Action	Works On
<code>.sum()</code>	Sum of values	DataFrame/Series
<code>.mean()</code>	Mean (average)	DataFrame/Series
<code>.count()</code>	Non-NA values	DataFrame/Series
<code>.size()</code>	All rows (including NA)	GroupBy only
<code>.min()</code> or <code>.max()</code>	Minimum/Maximum	DataFrame/Series
<code>.std()</code> or <code>.var()</code>	Standard deviation/Variance	DataFrame/Series
<code>.describe()</code>	Summary statistics (count, mean, std, etc.)	DataFrame/Series

E.g: Get Statistics

<code>s.sum()</code> , <code>df.sum()</code>	Sum of values
<code>s.cumsum()</code> , <code>df.cumsum()</code>	Cummulative sum of values
<code>s.min()/s.max()</code> , <code>df.min()/df.max()</code>	Minimum/maximum values
<code>s.idxmin()/series.idxmax()</code> , <code>df.idxmin()/df.idxmax()</code>	Minimum/Maximum index value
<code>s.describe()</code> , <code>df.describe()</code>	Summary statistics
<code>s.mean()</code> , <code>df.mean()</code>	Mean of values
<code>s.median()</code> , <code>df.median()</code>	Median of values

Applying Aggregation Functions Directly to a DataFrame

We can use different key statistical methods directly on a DataFrame or Series.

1. On Entire DataFrame (Numeric Columns Only)

python

`df.mean(numeric_only=True)` will avoid errors with string columns!

```
df.mean()  # Mean of all numeric columns
df.max()   # Max of all numeric columns
```

2. On a Single Column (Series)

python

```
df['Price'].sum()  # Sum of 'Price' column
df['Age'].std()    # Standard deviation of 'Age'
```

3. On Grouped Data (After `groupby()`) (Next Topic)

python

```
df.groupby('Category')['Price'].mean()  # Average price per category
```

```
import pandas as pd
```

```
# Sample DataFrame
```

```
data = {'Price': [10, 15, 20], 'Quantity': [3, 2, 5]}
```

```
df = pd.DataFrame(data)
```

```
# Direct aggregations
```

```
print("Sum of all numeric columns:")
```

```
print(df.sum())  # Output: Price 45, Quantity 10
```

```
print("\nMean of 'Price':")
```

```
print(df['Price'].mean())  # Output: 15.0
```

Pandas demo